scale

VeRO: An Evaluation Harness for Agents to Optimize Agents

Varun Ursekar, Apaar Shanker, Veronica Chatrath, Yuan (Emily) Xue, Sam Denton

Scale AI

varun.ursekar@scale.com

Abstract

An important emerging application of coding agents is *agent optimization*: the iterative improvement of a *target agent* through edit–execute–evaluate cycles. Despite its relevance, the community lacks a systematic understanding of coding agent performance on this task. Agent optimization differs fundamentally from conventional software engineering: the target agent interleaves deterministic code with stochastic LLM completions, requiring structured capture of both intermediate reasoning and downstream execution outcomes. To address these challenges, we introduce VERO (**Ver**sioning, **R**ewards, and **O**bservations), which provides (1) a reproducible evaluation harness with versioned agent snapshots, budget-controlled evaluation, and structured execution traces, and (2) a benchmark suite of target agents and tasks with reference evaluation procedures. Using VERO, we conduct an empirical study comparing optimizer configurations across tasks and analyzing which modifications reliably improve target agent performance. We release VERO to support research on agent optimization as a core capability for coding agents. Code is available at https://github.com/scaleapi/vero.

1. Introduction

LLM agents are programs that invoke language models and external tools to act in an environment [21, 24]. There are two broad approaches to improving an agent's performance: optimizing the LLM's weights through gradient-based learning such as Reinforcement Learning (RL), or optimizing the agent program itself.

For the latter, the design space depends on what one chooses to modify. Prompt optimization frameworks like DSPy [14] and TextGrad [31] restrict modifications to textual context, such as prompts and tool descriptions. However, automated optimization of the broader program structure (control flow, tool logic, and scaffolding) remains underexplored.

This problem is increasingly consequential as LLM agents have become the primary mechanism for deploying foundation models in production and realizing economic value. Yet, constructing effective agents in real-world settings remains a labor-intensive and largely manual optimization process: initialize the agent, observe failure modes from traces, refine prompts or tools, and iterate. This process is slow and unscalable as reliance on LLM-driven systems continues to grow [3, 27]. As demand for specialized agents accelerates, automating agent optimization becomes essential.

Simultaneously, there has been growing interest in *coding agents*: LLM agents equipped with tools for shell execution, code editing, and file operations [22, 26]. Benchmarks such as SWE-Bench [13] measure coding agents ability to perform real-world software engineering tasks. Similarly, MLEBench measure performance on classical machine learning engineering tasks [4]. While works such as AFlow [33] and ADAS [11] explore optimization of agents-as-code, they do not cast the optimization problem as an open-ended coding task for agents. To our knowledge, no standardized benchmark frames agent optimization as a code generation task and measures coding agents, acting as *optimizers*, on their ability to improve *target agents*.

Our work advances the understanding of coding agents as agent optimizers through three contributions:

1. **The VERO Harness.** An evaluation environment (Figure 1) providing versioned snapshots, budget-enforced evaluation, and structured execution traces. We show that without such infrastructure, optimization is unreliable: uncontrolled coding agents frequently fail to complete optimization runs or report unreproducible results.

Accepted to the Forty-Third International Conference on Machine Learning (ICML), 2026.

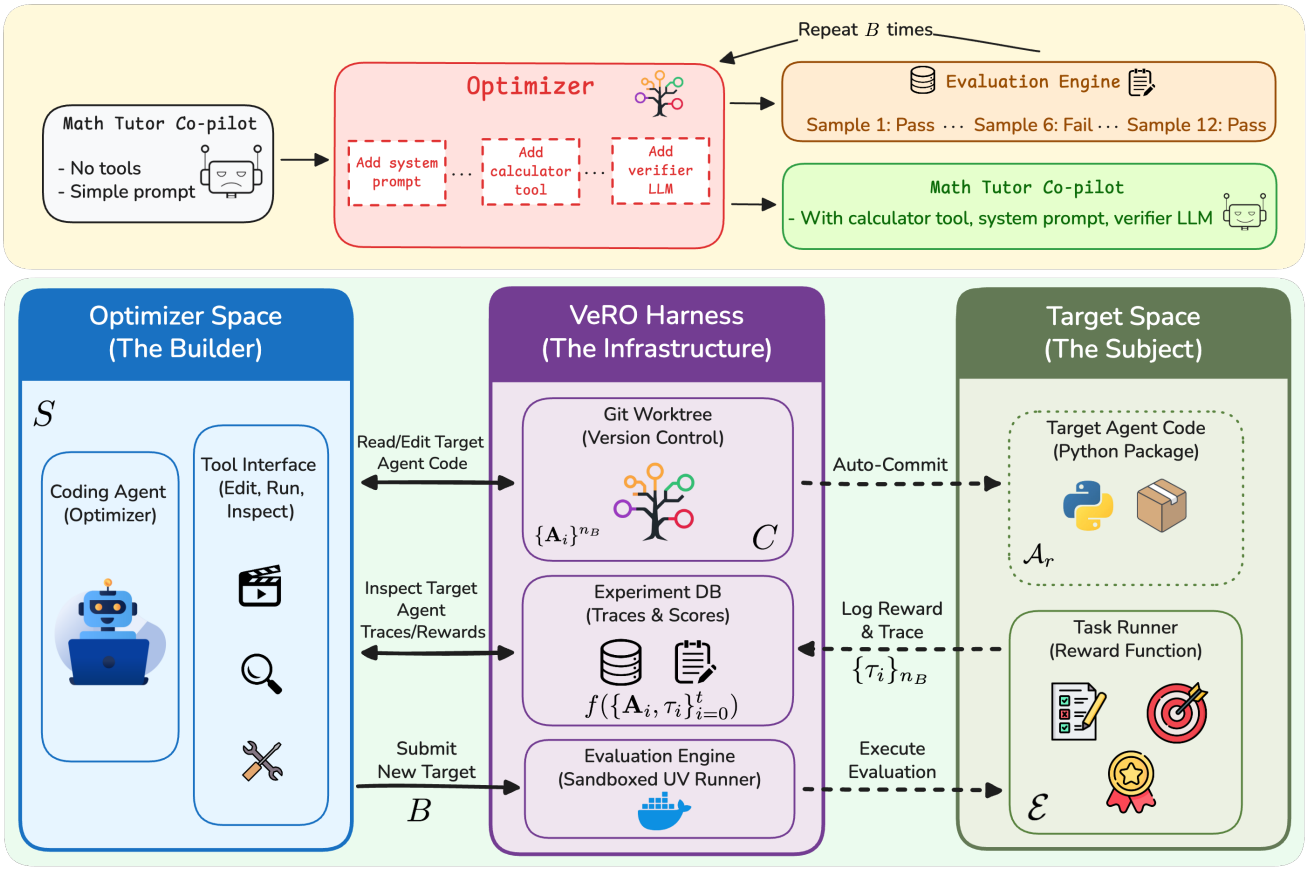


Figure 1: **VERO system architecture.** Top (orange): example optimization trajectory. Bottom (green): system components. VERO enforces versioning, reproducible execution, and controlled feedback, enabling systematic comparison of optimizers for agent optimization.

2. **The Agent Optimization Benchmark.** A standardized suite of target agents, tasks, and evaluation procedures spanning math reasoning, tool use, and multi-step QA. We report comprehensive results for state-of-the-art LLMs and coding agents, establishing initial baselines for the community.

3. **Empirical Findings.** Using VERO's tracing, we show that: (a) both minimal and sophisticated target agents admit meaningful optimization; (b) optimizer instructions strongly affect variance and cross-task generalization; (c) current optimizers default to prompt modifications, exhibiting limited diversity and impact in the changes they can produce.

2. Related Work

Optimization Using LLMs. A large number of works have applied LLMs to the tasks of black-box optimization and solution search. OPRO [25] motivates the application of LLMs to generic optimization problems via experiments on linear regression and the traveling salesman problem. FunSearch [20] and AlphaEvolve [17] apply LLMs and evolutionary search to discover novel algorithms. Self-Taught Optimizer (STOP) [32] extends code optimization to the agent's own code, demonstrating that LLMs can improve their own generation strategies. Robeyns et al. [19] also demonstrate the ability of LLM agents to improve themselves, but note the difficulty of stabilizing improvements.

Automated Agent Optimization. Several frameworks automate the design of agents and their components. Prompt optimization frameworks such as DSPy [14] automate the tuning of agent instructions and few-shot examples, but treat the underlying agent workflow as fixed. TextGrad [31], Trace [7], and AdalFlow [30] model agent workflows as computational graphs, using the resulting structure to propagate evaluation feedback for updating prompts and tools. ADAS [11] represents agents as functions in code and evaluates the ability of

agents to improve these functions on several benchmarks, including DROP [9] and GSM8K [8]. AFlow [33] formulates workflow generation as Monte Carlo Tree Search (MCTS) over code-represented graphs. Darwin Gödel Machine [34] explores open-ended agent evolution without fixed objectives.

Coding Agent Benchmarks. Coding agent evaluation has progressed from function-level synthesis (HumanEval [5], MBPP [2], LiveCodeBench [12] to repository-scale tasks requiring full environment access (SWE-Bench [13], TerminalBench [15]). We evaluate target agents on GAIA [16], GPQA [18], SimpleQA [23], TAU-Bench [29], and FACTS [6]. These benchmarks measure task completion, but treat agents as static artifacts. No existing benchmark frames agent optimization as a code generation task or measures coding agents on their ability to improve other agents.

3. Methodology

We investigate how LLM-based *target agents* can be optimized as code artifacts, with coding agents serving as the optimizers. Unlike prompt optimization, this framing treats the entire agent implementation—prompts, tools, and orchestration logic in a language like Python (alternately referred to as *workflow*)—as the optimization space. Here, we formalize the *target agent optimization task* and introduce VERO, a harness that provides both the execution infrastructure (isolated environments, resource constraints, guardrails) and evaluation protocols (versioned snapshots, structured feedback, reproducible measurement) necessary for enabling coding agents to perform this task.

3.1 Formal Problem Statement

We formalize the problem as follows. We define two distinct tasks: the *target agent task*, $\mathcal{T}$, which the target agent must solve, and *the optimization task*, $\mathcal{P}$, which is to improve the target agent's performance on $\mathcal{T}$.

A *target agent task*, $\mathcal{T} = (\mathcal{I}, \mathcal{O}, \mathcal{E})$ consists of: 1) An *input space*, $\mathcal{I}$, of task instances (e.g., questions, instructions, or problems the agent must address); 2) an *output space*, $\mathcal{O}$, comprising both the agent's final response and its execution trace (tool calls, intermediate reasoning steps); and 3) an *evaluation function*, $\mathcal{E} : \mathcal{O} \to [0, 1]$, that scores outputs, potentially using both the final response and the trace.

A *target agent*, $\mathbf{A} : \mathcal{I} \to \mathcal{O}$, is a program that maps task instances to outputs. We denote the space of all valid agent implementations as $\mathcal{A}$; in our setting, $\mathcal{A}$ consists of Python programs that invoke LLMs via tool-calling APIs. Importantly, both the agent, $\mathbf{A} \in \mathcal{A}$, and evaluation function, $\mathcal{E}$, may be stochastic due to LLM sampling. For each task, we assume access to a training set, $\mathcal{D}^{\text{train}} \subset \mathcal{I}$, and a held-out test set, $\mathcal{D}^{\text{test}} \subset \mathcal{I}$, drawn from a distribution of interest, $\mathcal{D}_{\mathcal{I}}$.

Optimization Task. Given a target agent task, $\mathcal{T}$, the goal is to find the agent that maximizes expected performance on held-out data:

$$\mathbf{A}^* = \underset{\mathbf{A} \in \mathcal{A}_r}{\text{argmax}} \quad \mathbb{E}_{x \sim \mathcal{D}^{\text{test}}, \mathcal{E}, \mathbf{A}}[\mathcal{E}(\mathbf{A}(x))]$$
$$\text{subject to} \quad n_{\mathcal{E}} \leq B$$

where $n_{\mathcal{E}}$ is the number of evaluation function calls, B is the maximum allowed budget, and $\mathcal{A}_r \subset \mathcal{A}$. The budget constraint reflects the cost of agent evaluation: each call requires executing the target agent and scoring its outputs, mirroring black-box optimization settings with expensive queries. The expectation is taken over the input distribution, the stochastic agent, $\mathbf{A}$, and the stochastic evaluator, $\mathcal{E}$ (both due to LLM sampling). Since these distributions are unknown, we control noise by fixing seeds where possible and averaging over samples. We denote this optimization problem as $\mathcal{P}$.

The search space, $\mathcal{A}_r$, is a *restricted* subspace of Python programs, where r encodes constraints: permitted model checkpoints, allowed APIs, SDK requirements, file-access permissions. These restrictions ensure fair comparison, prevent trivial solutions (e.g., upgrading to a stronger model), and ensure adherence to production constraints.

Finding $\mathbf{A}^*$ is intractable. Our practical objective is to maximize *lift*—the improvement over a baseline, $\mathbf{A}^{\text{base}}$:

$$\max_{\mathbf{A}^+ \in \mathcal{A}r} \mathbb{E}_{x \sim \mathcal{D}^{\text{train}}}[\mathcal{E}(\mathbf{A}^+(x)) - \mathcal{E}(\mathbf{A}^{\text{base}}(x))], \quad n_{\mathcal{E}} \leq B$$

This relative framing enables comparison across tasks with varying difficulty.

The Optimizer. We define the *coding agent*, S, as the optimizer that iteratively modifies the target agent, $\mathbf{A}$. At each step, t, S produces an updated implementation:

$$\mathbf{A}_{t+1} = S(f(\{\mathbf{A}_i, \tau_i\}_{i=0}^{t}),\ C)$$

where $\tau_t = \{(x_{tj}, o_{tj}, e_{tj})\}_{j=1}^{N}$ are *evaluation traces*—inputs, outputs, and scores from running $\mathbf{A}_t$ on training samples.

The Observation Function. The function, f, is the *observation interface*, specifying which aspects of the optimization history are exposed to S. This includes: which past agent versions are visible, how much trace detail is provided (full execution logs vs. summary statistics), and whether raw samples or only aggregate metrics are accessible. The design of f directly impacts the optimizer's ability to identify failure modes and develop effective modifications.

Finally, C denotes additional context provided to the optimizer: task descriptions, codebase documentation, and optimization guidance (e.g., best practices, common pitfalls). The interplay between S, f, and C is central to our experimental study.

3.2 A Protocol for Benchmarking Coding Agents as Agent Optimizers

To meaningfully compare coding agents on the optimization task, $\mathcal{P}$, an evaluation harness must satisfy three high-level goals: (a) *controlled comparison*—different optimizers, S, should operate under identical resource and environment conditions; (b) *informative feedback*—the optimizer must receive structured signals to guide search; and (c) *post-hoc interpretability*—a semantic understanding of the optimization trajectory should be fully recoverable for analysis.

These goals translate into six concrete requirements, each grounded in the formalism of Section 3.1:

1. Versioning. All modifications to the target agent must be captured as discrete snapshots (e.g., Git commits), yielding the sequence $\mathbf{A}_0, \mathbf{A}_1, \ldots, \mathbf{A}_T$. This enables rollback, `diff` inspection, and trajectory analysis.

2. Budget enforcement. The framework must enforce the constraint $n_{\mathcal{E}} \leq B$ by tracking evaluation calls and blocking requests that exceed the budget. This ensures optimizers cannot gain an advantage through additional compute.

3. Permission control. The restricted search space, $\mathcal{A}_r$, and context, C, must be programmatically enforced—limiting access to held-out test data, preventing model checkpoint changes, and restricting file-system writes. This operationalizes the constraints encoded in r.

4. Reproducible execution. Evaluations of a fixed agent version, $\mathbf{A}_t$, must be consistent. This requires dependency locking (e.g., via `uv` lockfiles) and environment isolation to minimize variance in $\mathbf{A}$ and $\mathcal{E}$.

5. Structured tracing. The evaluation traces, $\tau_t = \{(x_{tj}, o_{tj}, e_{tj})\}_j$, must capture sufficient detail—inputs, outputs, intermediate steps, and scores—to provide directional signal for the optimizer, S.

6. Standardized observation interface. The function, f, that exposes traces and history to the optimizer must be consistent across all optimizers being compared, ensuring no privileged access to information.

3.3 VERO: Versioning, Rewards, and Observations

VERO is a concrete instantiation of the protocol defined in Section 3.2. Figure 1 illustrates the architecture. We use a previously defined *coding agent* as the optimizer, S, to improve our target agent. The coding agent operates in a tool loop—generating actions, executing them, and observing results—until completion or budget exhaustion. The tool set, loop implementation, memory, and prompts together define the *agent scaffold*. Crucially, VERO is **agent scaffold-agnostic**: any coding agent that (i) consumes the interfaces exposed by VERO and (ii) preserves traceability of target-agent modifications (e.g., commit-level versioning) while enforcing resource and permission limits can be evaluated within the framework. For benchmarking, we also release a minimal coding-agent implementation as part of the framework, referred to as the VERO scaffold in our benchmark study in 4.1, with details in Appendix A.1.

3.3.1 Core Abstractions

VERO provides five abstractions that implement the outlined requirements. Each abstraction exposes functionality through *tools* (explicit interfaces the optimizer invokes) and *hooks* (transparent instrumentation). Tools provide

explicit interfaces to agents; hooks ensure consistent behavior across different scaffolds.

Git Worktree. The target agent codebase is a Git repository. VERO uses worktrees to isolate modifications, enabling parallel evaluation of commits. An *auto-commit hook* records file changes, producing an immutable trajectory in which `diffs` reveal exactly what changed and when. The *GitControl* tool provides access to version history and rollback.

Dataset. The Dataset abstraction manages inputs across splits, ($\mathcal{D}^{\text{train}}$, $\mathcal{D}^{\text{val}}$, $\mathcal{D}^{\text{test}}$). The *DatasetViewer* tool exposes samples from permitted splits while enforcing access control. The optimizer cannot view held-out test data.

Filesystem. The Filesystem abstraction enforces pattern-based access control over file operations. Rules constrains the optimizer to permitted regions of the codebase, preventing modifications to tests, ground-truth implementations, or evaluation infrastructure.

Experiment Database. All evaluation traces, τ_t, are stored in the Experiment Database: per-sample scores, errors, agent rollouts, and aggregate statistics. The *ExperimentViewer* exposes this data to the optimizer, providing structured feedback while maintaining audit trails.

Evaluator. The Evaluator executes target agents and computes metrics. The *ExperimentRunner* tool is a gated interface that enforces budget constraints: each request specifies a commit and samples; the engine checks out that version, runs evaluations, stores results, and decrements the budget. This ensures no optimizer gains advantage through additional compute.

3.3.2 Fair Comparison Across Optimizers

Hooks provide standardization. Different optimizers may use different tools (e.g., Claude's native tools vs. OpenAI Agents SDK). However, VERO operates below the tool layer, ensuring consistent behavior regardless of interfaces: file modifications trigger auto-commits, evaluations go through the gated *Evaluator*, and data access is mediated by viewers, meaning optimizers can be compared fairly even if their tool interfaces differ.

Target agents are reproducible packages. Each target agent is structured as a `uv`-managed Python package with a lockfile that pins all dependencies. Combined with Git versioning, this ensures any commit can be re-evaluated consistently (same code, dependencies, execution environment). This mirrors SWE-bench's uses Docker containers to ensure reproducible patch evaluation [13].

3.3.3 Optimization Loop

VERO does not prescribe a search strategy; the optimizer retains full autonomy. Algorithm 1 illustrates a typical loop, where each iteration yields $\mathbf{A}_{t+1} = S(f(\{\mathbf{A}_i, \tau_i\})_{i=0}^{t}, C)$, with the tools collectively implementing f.

4. Experimental Setup

We conduct three complementary studies using VERO: (1) a **benchmark study** comparing optimizer configurations across multiple tasks, (2) a **robustness study** examining how performance improvements translate across model families, and (3) a **case study** analyzing optimization behavior on two base target agents of differing complexity. We provide interpretability analyses of these studies in both the Results and Appendix. Optimizer models always use a temperature of 1.0. We select the best commit per run based on validation performance.

4.1 Benchmark Study

Tasks. We evaluate on five tasks spanning math reasoning (MATH [10]), tool use (TAU-Bench Retail [29]), multi-step reasoning (GAIA [16]), factual QA (SimpleQA), and science QA (GPQA). Table 1 shows split sizes for each dataset, along with the tools provided to the initial target agent. The prompts for the target agent are hand-crafted and task-specific. The target agent model is always set to GPT-4.1 mini.

Protocol. We compare five optimizer configurations, averaging over $N = 3$ iterations, yielding 105 total experiments (Table 2). We set the budget $B = 8$ for all iterations. Each configuration consists of a coding agent *scaffold*, a *variant* of the scaffold (e.g. specific selections of tools), and an optimizer *model*. VERO scaffold variants include: *Default* (full tools with sub-agent delegation and access to a an agent design pattern library – which we define as

Algorithm 1 Typical VERO Optimization Loop

Require: Base agent $\mathbf{A}_0$, budget B, context C

$t \leftarrow 0, n_{\mathcal{E}} \leftarrow 0$
while $n_{\mathcal{E}} < B$ **do**
  // Inspect
  $\mathcal{D} \leftarrow$ DATASETVIEWER.GETSAMPLES()
  $\tau_{<t} \leftarrow$ EXPERIMENTVIEWER.GETTRACES()
  $\text{code}_t \leftarrow$ FILETOOLS.READ($\mathbf{A}_t$)
  // Hypothesize & Implement
  $\Delta \leftarrow S(\mathcal{D}, \tau_{<t}, \text{code}_t, C)$ *// LLM proposes edit*
  FILETOOLS.WRITE(Δ) *// auto-commit hook fires*
  $\mathbf{A}_{t+1} \leftarrow$ GITCONTROL.GETHEAD()
  // Evaluate
  $\tau_{t+1} \leftarrow$ EXPERIMENTRUNNER.RUN($\mathbf{A}_{t+1}$)
  $n_{\mathcal{E}} \leftarrow n_{\mathcal{E}} + 1$
  // Iterate
  if $\text{score}(\tau_{t+1}) < \text{score}(\tau_t)$ **then**
    GITCONTROL.ROLLBACK($\mathbf{A}_t$) *// optional*
  end if
  $t \leftarrow t + 1$
end while
return $\arg\max_i \text{score}(\tau_i)$

a *Cookbook*), *Orchestrator* (sub-agent delegation bias), and *Resources-Only* (restricted to modifications of prompts, tool descriptions, and parameters). Claude Code scaffold variants include: VERO *Tools* (with VERO tools and hooks for evaluation invocations, trace/dataset access, and pattern library access), and *Pure* (no access to VERO tools). By default, we use Claude Sonnet 4.5 as the optimizer model (75 runs), except for 2 VERO variants where we ablate the underlying LLM with Claude Opus 4.5 (15 runs) and GPT-5.2-Codex (15 runs). We control for the optimizer prompt across configurations. Details on each configuration are provided in Appendix A.2.1.

4.2 Robustness Study

We evaluate whether performance changes resulting from modifications to the target agent by the optimizer transfer when the target model used during optimization is replaced by another model. We extract the best performing commits found by *Orchestrator* variants using Sonnet and GPT-5.2 for the GAIA, GPQA, SimpleQA, and TAU-Bench Retail tasks. These commits are re-evaluated with the target agent model substituted with Claude Sonnet-4.5, GPT-4.1, Gemini 2.5 Flash, and Qwen3 variants, representing varying relationships with respect to both the optimizer model and the original target agent model.

Task	Domain	Tr/Val/Te	Initial Tools
GAIA	Multi-step	50/87/—	—
GPQA	Science QA	98/—/100	—
MATH	Math	59/60/486	—
TAU-Bench Retail	Tool use	100/20/115	Original
SimpleQA	Factual QA	46/45/80	Wikipedia

Table 1: Benchmark study tasks with respective Train / Validation / Test data splits and *base agents*.

4.3 Case Study: GAIA with Realistic Agents

To study optimization dynamics beyond simple baselines, we conduct a controlled experiment on GAIA using two agents with varying capability: **Pawn** (minimal): 4 tools, 25-line prompt, standard library only, 20 max turns and **Knight** (sophisticated): 6 tools (incl. Wikipedia, LLM-powered reflection), 140-line prompt with ReACT [28] patterns, extended libraries (pandas, numpy), complex file support, 40 max turns. Refer to Section A.3.1 of the Appendix for further details.

Scaffold	Variant	Model	GAIA	GPQA	MATH	Retail	SimpleQA	Avg.
Baseline	—	—	0.07	0.60	0.87	0.38	0.61	0.50
Claude Code	Pure	Sonnet	0.13 (0.29)	0.58 (0.63)	**0.88** (**0.90**)	0.43 (0.46)	0.65 (0.68)	0.53 (0.59)
Claude Code	VERO Tools	Sonnet	0.14 (0.21)	0.64 (**0.71**)	**0.88** (**0.90**)	0.39 (0.43)	0.67 (0.71)	0.55 (0.59)
VERO	Default	Sonnet	**0.26** (**0.30**)	0.64 (0.65)	0.86 (0.87)	**0.55** (0.66)	0.73 (0.76)	**0.61** (**0.65**)
VERO	Orchestrator	Opus	0.18 (0.18)	0.62 (0.66)	**0.88** (0.88)	**0.55** (0.57)	**0.74** (**0.86**)	0.59 (0.63)
VERO	Orchestrator	Sonnet	0.16 (0.20)	0.62 (0.65)	0.87 (0.88)	0.51 (**0.72**)	0.71 (0.72)	0.57 (0.63)
VERO	Orchestrator	GPT-5.2	0.07 (0.09)	**0.65** (0.70)	**0.88** (**0.90**)	0.40 (0.42)	0.60 (0.62)	0.52 (0.55)
VERO	Resources Only	Sonnet	0.11 (0.13)	0.60 (0.64)	**0.88** (0.88)	0.42 (0.43)	0.69 (0.72)	0.54 (0.56)

Table 2: **Benchmark Suite Results.** Each cell reports the average best score over $N = 3$ iterations, with maxima in parentheses. The baseline shows average initial-commit performance over $t \times 3 = 21$ runs. GPT-4.1 mini is used as the target agent model throughout.

This pairing tests two hypotheses: whether optimizers can (1) *expand* a minimal agent's capabilities, and (2) *refine* an already-sophisticated agent.

Independent Variable. We vary the *instruction templates* provided to the optimizer, controlling how much guidance the coding agent receives. Templates differ in degree and style of guidance, as detailed in Appendix A.3.2.

Protocol. Each configuration is run $N = 4$ times with a budget $B = 5$ on the GAIA training split. We evaluate on three held-out sets, GAIA validation (87 samples), FACTS Search [6] (890 samples), and SimpleQA (45 samples), to assess both in-distribution improvement and cross-task generalization.

Metrics. We report: (1) *lift*: maximum accuracy gain over baseline across iterations; (2) *variance*: standard deviation across iterations, measuring optimization stability; and (3) *runtime*: mean inference time per sample, capturing efficiency tradeoffs.

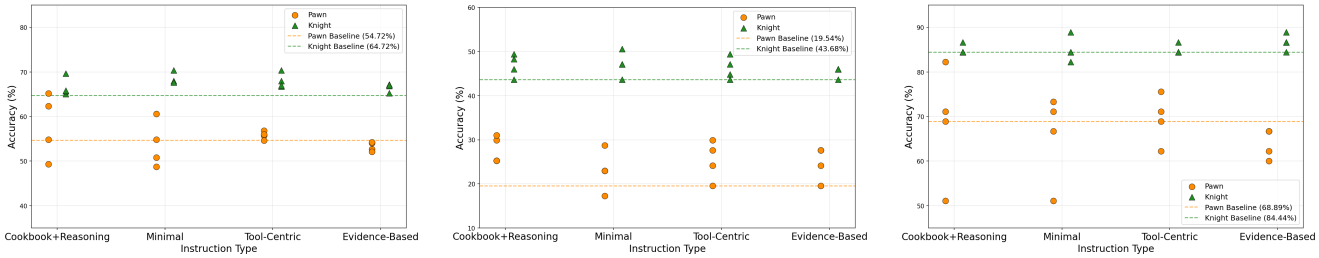
5. Results and Discussion

5.1 Benchmark Study Results

Table 2 presents the main results across all optimizer configurations and tasks. For each task, we report the average initial score (*Baseline*), average best score across $N = 3$ iterations, and the maximum best score in parentheses. In Appendix Figure 5, we show a visualization of the lift achieved by each optimizer configuration across all tasks.

VERO Provides the Necessary Harness for Real Gains. Claude Code *Pure* shows limited improvements relative to VERO-enabled variants. Holding the LLM constant (Sonnet), adding VERO tools increases average (best) performance by 2% (0%), while the full VERO harness yields gains of 8% (6%). This highlights our key contribution: VERO is necessary for the agent-building task. Within the VERO variants, performance follows a clear hierarchy, with *Default* outperforming *Orchestrator* and *Resources Only*.

Optimizer success is highly task-dependent. While all configurations show some average improvement over baseline, gains vary substantially by task. Reasoning-heavy tasks (GPQA, MATH), exhibit little to no improvement across configurations, whereas tool-use-oriented tasks (GAIA, Retail, SimpleQA) show consistent gains. *Resources Only* underperforms most consistently, indicating these tasks benefit from richer scaffolding changes beyond prompt changes.



(a) FACTS (b) GAIA (c) SimpleQA

Figure 2: Case study results highlighting optimization outcomes across optimizer instructions types for FACTS, GAIA, and SimpleQA. Orange circles: Pawn agent; green triangles: Knight agent. Dashed lines: baseline accuracies.

Choice of optimizer model affects outcomes, but no single model dominates. Among *Orchestrator* configurations, ablating the optimizer LLM shows that Claude models (Sonnet, Opus) significantly outperform GPT-5.2-Codex on GAIA, Retail, and SimpleQA, while GPT-5.2 performs best on GPQA. Notably, Sonnet achieves the highest maximum score on TAU-Bench Retail, outperforming its larger sibling. Overall, optimizer model performance is task-dependent, with non-trivial effects of both model family and size.

5.2 Robustness Study Results

Performance gains persist for a target model in the same model family. Using the best checkpoints created from our optimization runs against GPT-4.1 mini in Table 2, we see that across both *optimizer model* configurations,

Task	Optimizer	Target Model	Init.	Final	Δ
GAIA	Codex	GPT-4.1	0.15	0.11	-0.03
	Codex	Gemini-Flash	0.26	0.24	-0.02
	Codex	Qwen3-30B	0.11	0.03	-0.08
	Codex	Qwen3-4B	0.08	0.07	-0.01
	Sonnet	GPT-4.1	0.15	0.22	+0.07
	Sonnet	Gemini-Flash	0.26	0.21	-0.06
	Sonnet	Qwen3-30B	0.11	0.16	+0.05
	Sonnet	Qwen3-4B	0.08	0.16	+0.08
GPQA	Codex	Claude-Sonnet	0.71	0.74	+0.03
	Codex	GPT-4.1	0.62	0.68	+0.06
	Sonnet	Claude-Sonnet	0.71	0.66	-0.05
	Sonnet	GPT-4.1	0.62	0.64	+0.02
	Sonnet	Qwen3-30B	0.52	0.56	+0.04
	Sonnet	Qwen3-4B	0.52	0.45	-0.07
SimpleQA	Codex	GPT-4.1	0.75	0.76	+0.01
	Codex	Gemini-Flash	0.68	0.70	+0.02
	Sonnet	GPT-4.1	0.74	0.76	+0.02
	Sonnet	Gemini-Flash	0.66	0.66	+0.00
TAU-Bench Retail	Codex	Claude-Sonnet	0.57	0.56	-0.02
	Codex	GPT-4.1	0.52	0.50	-0.03
	Sonnet	Claude-Sonnet	0.59	0.81	+0.22
	Sonnet	GPT-4.1	0.55	0.79	+0.24

Table 3: **Robustness Study Results.** Evaluation results on the best commits found by *Orchestrator* variants in Table 2. Results show performance of various model checkpoints with target agents that were optimized with GPT-4.1 mini as the target model.

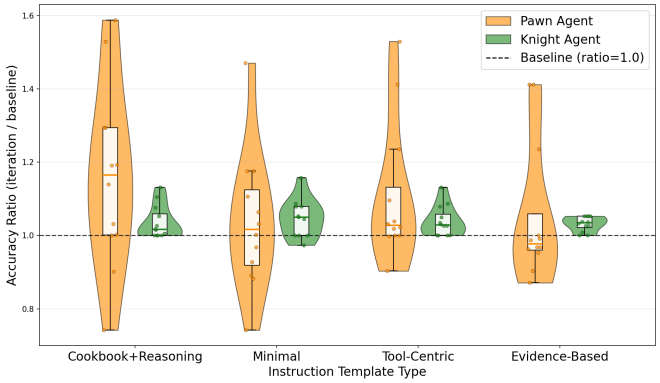


Figure 3: Accuracy ratio (iteration accuracy/baseline) across instruction types, aggregated over all three evaluation datasets (FACTS, GAIA, SimpleQA) for Pawn and Knight agents.

GPT-4.1 sees varying positive performance gains across tasks; for the two rows in which we see negative outcomes, i.e. where Codex optimizes agents for GAIA and TAU-Bench Retail, we note that the original optimization on GPT-4.1 mini yielded no gains; this failure is reflected when altering the target model.

Strong positive changes do not always generalize and generalization depends on both model and task. Among the *Orchestrator* Sonnet configurations in the benchmark study in Table 2, the major gains were on GAIA (+0.13), GPQA (+0.05), SimpleQA (+0.11), and TAU-Bench Retail (+0.28). Across these tasks, we see a range of transferability when run against new target models including some significant sustained gains on GAIA and TAU-Bench Retail with regressions in performance on out-of-family target models such as Gemini-Flash and Qwen3-4B.

5.3 Case Study: Optimization on Realistic Agents

Figure 2 shows optimization outcomes across four instruction templates for **Pawn** and **Knight** agents, with each point representing one of four optimization iterations.

Optimization headroom inversely correlates with *agent complexity*. Figure 2 reveals an asymmetry. Optimizations of the Pawn exhibit larger maximum lifts: +11.5% on GAIA, +10.5% on FACTS, and +13.3% on SimpleQA. Knight shows more modest gains: +6.9% on GAIA, +5.6% on FACTS, and +4.5% on SimpleQA. This suggests that optimization headroom depends critically on the target agent's initial sophistication. Intuitively, simpler agents present more opportunities for improvements (adding tools, expanding capabilities), than sophisticated agents.

Optimizer instructions pair tightly with the base target agent. We observe that the optimal instruction template differs by agent sophistication. As shown in Figure 3, *Cookbook+Reasoning* achieves the highest mean accuracy for Pawn averaged across all three evaluation datasets. Here, detailed guidance helps the optimizer uncover structural improvements on a minimal base agent. Conversely, Knight performs best under *Minimal*, which

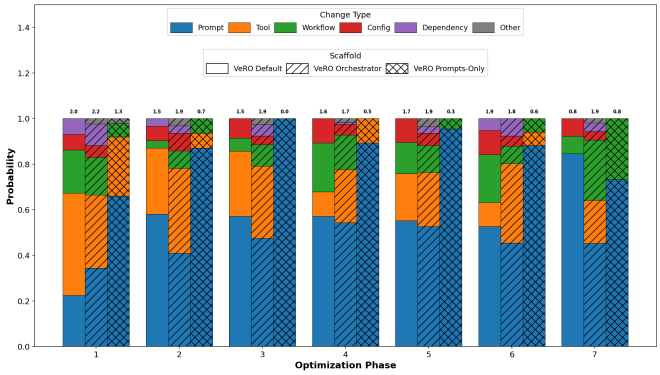


Figure 4: We plot the probability of changes made between successive evaluations in a trajectory falling into one of several types (e.g. prompt, tool, workflow changes) across all trajectories conducted using VERO scaffolds. **Bold** numbers capture the entropy of the probability distribution defined by each bar.

removes cookbook access and reduces prompt size by 65% (Appendix A.3.2). This counterintuitive result suggests that prescriptive guidance can constrain already capable agents: Knight's sophistication benefits more from creative freedom than from following established patterns. Template selection should therefore reflect the target agent's baseline capabilities.

Optimizer instructions induce a variance–performance tradeoff. Figure 3 reveals a consistent pattern for both agents: high-variance templates yield higher peak performance, while low-variance templates sacrifice upside for consistency. For **Pawn**, *Cookbook+Reasoning* produces the highest variance and also the highest mean accuracy gain. In contrast, *Tool-Centric* and *Evidence-Based* yield more stable trajectories but fail to reach the same peaks. The same pattern holds for **Knight**: *Minimal* shows elevated variance alongside highest mean accuracy gain, while *Evidence-Based* achieves the lowest variance but caps potential gains. This tradeoff stems directly from it's design where explicit anti-patterns (e.g., "avoid adding complex new tools") and single-variable experimentation constraints prevent high-risk modifications that may yield breakthroughs.

Generalization across evaluation sets is not guaranteed. Optimizations targeting one task do not uniformly transfer to related tasks. Under *Cookbook+Reasoning*, Pawn iteration 3 achieved +5.75% on GAIA but simultaneously regressed -17.8% on SimpleQA (Appendix A.3.4). Inspection of the commit reveals the optimizer added a complex multi-step verification tool that improved multi-hop reasoning but introduced unnecessary overhead for simple factual queries. This pattern, where task-specific optimizations harm performance elsewhere, underscores the importance of validating on multiple tasks.

Runtime efficiency varies substantially across templates. Beyond accuracy, templates affect the computational cost of optimized agents. *Evidence-Based* produces agents that run 2× faster than *Tool-Centric* (26.2s vs. 56.6s per sample for Knight; 12.6s vs. 32.8s for Pawn) while maintaining competitive accuracy (Appendix A.3.1). This efficiency gain stems from the template's emphasis on lightweight modifications and explicit discouragement of complex tool additions. While we do impose timeouts on target agent rollouts, we do not explicitly factor this into our budget definition B; we discuss this limitation in Appendix A.4.

5.4 Interpretability

We leverage Git commit histories of the coding agent with persisted target agent traces and rewards to investigate semantic trends in the optimization process for different tasks. A wholistic set of interpretability analyses are available in Appendix A.2.3 as well as annotated sample trajectories with key optimizer contributions in Appendix A.2.4.

Method. We use GPT-4.1 to tag changes made by the coding agent during each optimization trajectory with one or more pre-defined tags. We define each series of changes between successive invocations of the evaluation engine as an optimization *phase*, capturing the intuition that these constitute one attempted strategy by the coding agent. For more details of how we extract phases, we refer the reader to Appendix A.2.3. In Figure 4, we plot the frequency of these semantic tags across all runs with the VERO scaffold in Table 2 as a function of optimization phase grouped by the three VERO variants. We highlight three key findings:

Prompt modifications dominate. The coding agent configurations we tested attempt prompt changes over 50% of the time in all phases beyond the first, consistently across configurations and tasks. However, we do see that coding agents have the ability to increase their focus on other features, such as tools, when the task demands it (as shown for GAIA in Figure 6 in Appendix A.2.3).

Scaffold biases influence diversity of change types. The diversity of changes, as measured by entropy of the tag distribution within each phase, made by the *Orchestrator* variant is almost always larger than that of the other two. As expected, the *Resources Only* has extremely low entropy across phases due to its constraints.

Entropy of changes decreases over phases. Across all three variants, diversity drops sharply after the first phase, suggesting that coding agents often revert to prompt changes when their more ambitious modifications fail to yield gains. This decline is less pronounced for the *Orchestrator* variant. Figure 9 in Appendix A.2.4 plots entropy across phases for all five benchmarks, further illustrating this trend.

6. Limitations

The limitations of this work are detailed in Appendix A.4. Chief among these: (1) budget is specified only in evaluation calls, not tokens or API cost, introducing variance; (2) we do not ablate budget size or compare

multi-phase versus single-phase optimization; (3) we lack human baselines for target agents; and (4) public API instability and potential reward hacking via leaked ground truth are not controlled. These limitations point to important directions for strengthening the evaluation methodology.

7. Conclusion

VERO is a harness for evaluating coding agents on agent optimization: iteratively improving target agents through code modifications. We expose critical gaps: current optimizers lack diversity, defaulting to prompt edits over structural changes; improvements on tool-use tasks fail to transfer to reasoning domains; and instruction sensitivity remains poorly understood. These findings position agent optimization as a capability frontier, tractable but far from solved. We release VERO to enable the community to benchmark progress and develop models that close these gaps.

Impact Statement

This paper presents work whose goal is to advance the field of Machine Learning by formalizing the autonomous optimization of LLM agents. By introducing the VERO framework and a standardized benchmark for the agent-building task, we aim to lower the barrier for creating robust, self-improving systems.

The potential societal consequences of this work are as follows: On the positive side, automating the "Agent-as-Code" optimization loop can significantly accelerate the development of AI tools for scientific research, education, and accessibility, making high-performance agentic systems more efficient and less reliant on manual engineering.

However, we recognize the ethical considerations inherent in self-evolving code. The ability for agents to autonomously modify their own workflows and tool-use logic necessitates rigorous safety guardrails to prevent unintended behaviors or the circumvention of original safety constraints. Furthermore, as agents become more adept at building and refining other agents, the transparency and interpretability of these "machine-authored" systems become critical. We encourage the community to use the VERO benchmark and harness not only to improve performance but also to develop robust verification methods for autonomous code evolution.

References

[1] Anthropic. Claude 3.7 Sonnet and Claude Code. URL https://www.anthropic.com/news/claude-3-7-sonnet.

[2] J. Austin, A. Odena, M. Nye, M. Bosma, H. Michalewski, and et al. Program synthesis with large language models, 2021. URL https://arxiv.org/abs/2108.07732.

[3] Boston Consulting Group AI Platforms Group. Building effective enterprise agents. Technical report, Boston Consulting Group, November 2025. URL https://www.bcg.com/assets/2025/building-effective-enterprise-agents.pdf.

[4] J. S. Chan, N. Chowdhury, O. Jaffe, J. Aung, D. Sherburn, E. Mays, G. Starace, K. Liu, L. Maksin, T. Patwardhan, A. Madry, and L. Weng. MLE-bench: Evaluating machine learning agents on machine learning engineering. In *The Thirteenth International Conference on Learning Representations*, 2025. URL https://openreview.net/forum?id=6s5uXNWGIh.

[5] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P. de Oliveira Pinto, and et al. Evaluating large language models trained on code, 2021. URL https://arxiv.org/abs/2107.03374.

[6] A. Cheng, A. Jacovi, A. Globerson, B. Golan, C. Kwong, and et al. The facts leaderboard: A comprehensive benchmark for large language model factuality, 2025. URL https://arxiv.org/abs/2512.10791.

[7] C.-A. Cheng, A. Nie, and A. Swaminathan. Trace is the new autodiff — unlocking efficient optimization of computational workflows. In *Automated Reinforcement Learning: Exploring Meta-Learning, AutoML, and LLMs*, 2024. URL https://openreview.net/forum?id=Lg4AnAFsMd.

[8] K. Cobbe, V. Kosaraju, M. Bavarian, M. Chen, H. Jun, and et al. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.

[9] D. Dua, Y. Wang, P. Dasigi, G. Stanovsky, S. Singh, and M. Gardner. DROP: A reading comprehension benchmark requiring discrete reasoning over paragraphs. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, Minneapolis, Minnesota, June 2019. Association for Computational Linguistics. doi: 10.18653/v1/N19-1246. URL https://aclanthology.org/N19-1246/.

[10] D. Hendrycks, C. Burns, S. Kadavath, A. Arora, S. Basart, and et al. Measuring mathematical problem solving with the math dataset. *arXiv preprint arXiv:2103.03874*, 2021.

[11] S. Hu, C. Lu, and J. Clune. Automated design of agentic systems. In *The Thirteenth International Conference on Learning Representations*, 2025. URL https://openreview.net/forum?id=t9U3LW7JVX.

[12] N. Jain, K. Han, A. Gu, W.-D. Li, F. Yan, and et al. Livecodebench: Holistic and contamination free evaluation of large language models for code. In *The Thirteenth International Conference on Learning Representations*, 2025. URL https://openreview.net/forum?id=chfJJYC3iL.

[13] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, and et al. SWE-bench: Can language models resolve real-world github issues? In *The Twelfth International Conference on Learning Representations*, 2024. URL https://openreview.net/forum?id=VTF8yNQM66.

[14] O. Khattab, A. Singhvi, P. Maheshwari, Z. Zhang, K. Santhanam, and et al. DSPy: Compiling declarative language model calls into self-improving pipelines. In *The Twelfth International Conference on Learning Representations*, 2024.

[15] M. A. Merrill, A. G. Shaw, N. Carlini, B. Li, H. Raj, and et al. Terminal-bench: Benchmarking agents on hard, realistic tasks in command line interfaces, 2026. URL https://arxiv.org/abs/2601.11868.

[16] G. Mialon, C. Fourrier, C. Swift, T. Wolf, Y. LeCun, and T. Scialom. Gaia: a benchmark for general ai assistants, 2023. URL https://arxiv.org/abs/2311.12983.

[17] A. Novikov, N. Vũ, M. Eisenberger, E. Dupont, P.-S. Huang, and et al. Alphaevolve: A coding agent for scientific and algorithmic discovery, 2025. URL https://arxiv.org/abs/2506.13131.

[18] D. Rein, B. L. Hou, A. C. Stickland, J. Petty, R. Y. Pang, and et al. GPQA: A graduate-level google-proof q&a benchmark. In *First Conference on Language Modeling*, 2024. URL https://openreview.net/forum?id=Ti67584b98.

[19] M. Robeyns, M. Szummer, and L. Aitchison. A self-improving coding agent, 2025. URL https://arxiv.org/abs/2504.15228.

[20] B. Romera-Paredes, M. Barekatain, A. Novikov, M. Balog, M. P. Kumar, and et al. Mathematical discoveries from program search with large language models. *Nature*, 2023. doi: 10.1038/s41586-023-06924-6.

[21] L. Wang, C. Ma, X. Feng, Z. Zhang, H. Yang, J. Zhang, Z. Chen, J. Tang, X. Chen, Y. Lin, et al. A survey on large language model based autonomous agents. *Frontiers of Computer Science*, 18(6):186345, 2024.

[22] X. Wang, B. Li, Y. Song, F. F. Xu, X. Tang, and et al. OpenHands: An Open Platform for AI Software Developers as Generalist Agents. URL https://arxiv.org/abs/2407.16741.

[23] J. Wei, N. Karina, H. W. Chung, Y. J. Jiao, S. Papay, and et al. Measuring short-form factuality in large language models, 2024. URL https://arxiv.org/abs/2411.04368.

[24] L. Weng. Llm powered autonomous agents. *lilianweng.github.io*, 2023. URL https://lilianweng.github.io/posts/2023-06-23-agent/.

[25] C. Yang, X. Wang, Y. Lu, H. Liu, Q. V. Le, D. Zhou, and X. Chen. Large language models as optimizers. In *The Twelfth International Conference on Learning Representations*, 2024. URL https://openreview.net/forum?id=Bb4VGOWELI.

[26] J. Yang, C. E. Jimenez, A. Wettig, K. Lieret, S. Yao, and et al. SWE-agent: Agent-computer interfaces enable automated software engineering. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*, 2024. URL https://arxiv.org/abs/2405.15793.

[27] J. Yang, N. Yonack, K. Zyskowski, D. Yarats, J. Ho, and J. Ma. The adoption and usage of ai agents: Early evidence from perplexity. Technical Report Working Paper 26-040, Harvard Business School, 2025. URL https://www.hbs.edu/ris/Publication%20Files/26-040_ac431922-9f75-4f7d-b6dc-b67bb1c02c50.pdf.

[28] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, and et al. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations*, 2023.

[29] S. Yao, N. Shinn, P. Razavi, and K. R. Narasimhan. τ-bench: A benchmark for Tool-Agent-User interaction in real-world domains. In *The Thirteenth International Conference on Learning Representations*, 2025. URL https://openreview.net/forum?id=roNSXZpUDN.

[30] L. Yin and Z. Wang. Llm-autodiff: Auto-differentiate any llm workflow, 2025. URL https://arxiv.org/abs/2501.16673.

[31] M. Yuksekgonul, F. Bianchi, J. Boen, S. Liu, P. Lu, and et al. Optimizing generative ai by backpropagating language model feedback. *Nature*, 639:609–616, 2025. doi: 10.1038/s41586-025-08661-4.

[32] E. Zelikman, E. Lorch, L. Mackey, and A. T. Kalai. Self-taught optimizer (stop): Recursively self-improving code generation. In *Conference on Language Modeling (CoLM)*, 2024. URL https://arxiv.org/abs/2310.02304.

[33] J. Zhang, J. Xiang, Z. Yu, F. Teng, X.-H. Chen, and et al. AFlow: Automating agentic workflow generation. In *The Thirteenth International Conference on Learning Representations*, 2025. URL https://openreview.net/forum?id=z5uVAKwmjf.

[34] J. Zhang, S. Hu, C. Lu, R. Lange, and J. Clune. Darwin gödel machine: Open-ended evolution of self-improving agents. In *The Fourteenth International Conference on Learning Representations*, 2026. URL https://openreview.net/forum?id=pUpzQZTvGY.

A. Appendix

A.1 VERO Scaffold

As mentioned in Section 3.3, VERO as a framework is largely agnostic to the coding agent scaffold. In our experiments throughout this work, however, we use our own minimal implementation of a coding agent scaffold – the VERO scaffold – which we make available as part of the framework. We describe the scaffold here.

Architecture. The VERO scaffold is implemented as a Python class that serves both as a container for tools tied to the main abstractions in the VERO framework and an executor of LLM completions and tool logic. The class configuration consists of a selection of tools, configurations of selected tools, as well as settings for LLM completions. The VERO variants we describe in Section 4.1 are essentially different configurations of this class. Key architectural highlights of the scaffold include:

- **Tools Derived From Core Abstractions:** Tools are first-class objects instantiated with dependency injection of core abstractions (filesystem, worktrees, datasets, experiment database, and evaluator).
- **Access control:** The `Filesystem` abstraction enforces fine-grained read/write permissions via glob-style patterns, while dataset splits have explicit access levels. These can be leveraged to configure tool in other scaffolds.
- **Git Versioning:** The `GitWorktree` abstraction enables the construction of a number of tools, while preserving observability into changes agents make via hooks, e.g. auto-commit.

Toolsets. The scaffold provides a comprehensive set of tools that enable file reading/writing, trace viewing, dataset viewing, and target agent evaluation invocation. Table 4 describes the available toolsets.

A.2 Benchmark Study: Supplementary Materials

Here we provide several supplementary details and analyses pertaining to our benchmark study Benchmark Study.

A.2.1 Optimizer Configurations

We evaluate two scaffolds: the **VERO scaffold**, our minimal implementation described in Section A.1, and the **Claude Code scaffold**, Anthropic's agentic coding system [1]. Claude Code provides a complete coding agent with file editing, terminal access, and web search, with the ability to use hooks to restrict the behaviour of tools and trigger actions pre- and post-tool use (e.g. auto-committing). We integrate VERO abstractions into Claude Code via MCP tools and bash hooks. Table 5 summarizes the five configurations tested. Each configuration combines a scaffold, a variant (tool/mode selection), and an optimizer model.

A.2.2 Benchmarking Results

Table 2 shows the average score of the best commits found by different optimizer configurations on the 5 tasks in our benchmark suite. In Figure 5, we show the average *lift* per configuration on each task as a visual depiction of the contents of the table. We highlight that across configurations we observe around 8-9% lift on GAIA, TAU-Bench Retail, and Simple QA, tasks that require the use of tools. In contrast, GPQA and MATH show almost no gains across configurations.

Mean Normalized Optimization Phases. In Figure 7, we show the mean normalized *phase* at which the optimal agent version is found. Recall that the coding agent S is equipped with a tool to evaluate the target agent $\mathbf{A}$ on the training set. It may make several commits in between successive invocations of this tool. We define a sequence of such commits between evaluation as a *phase*: a sequence of commits that represents a logical change. Figure 7 shows that for TAU-Bench Retail, GAIA, Simple QA the best commit is often found before the halfway point in the trajectory. This could be an indicator that solution agents for these optimization tasks are easier to find compared to those of MATH and GPQA.

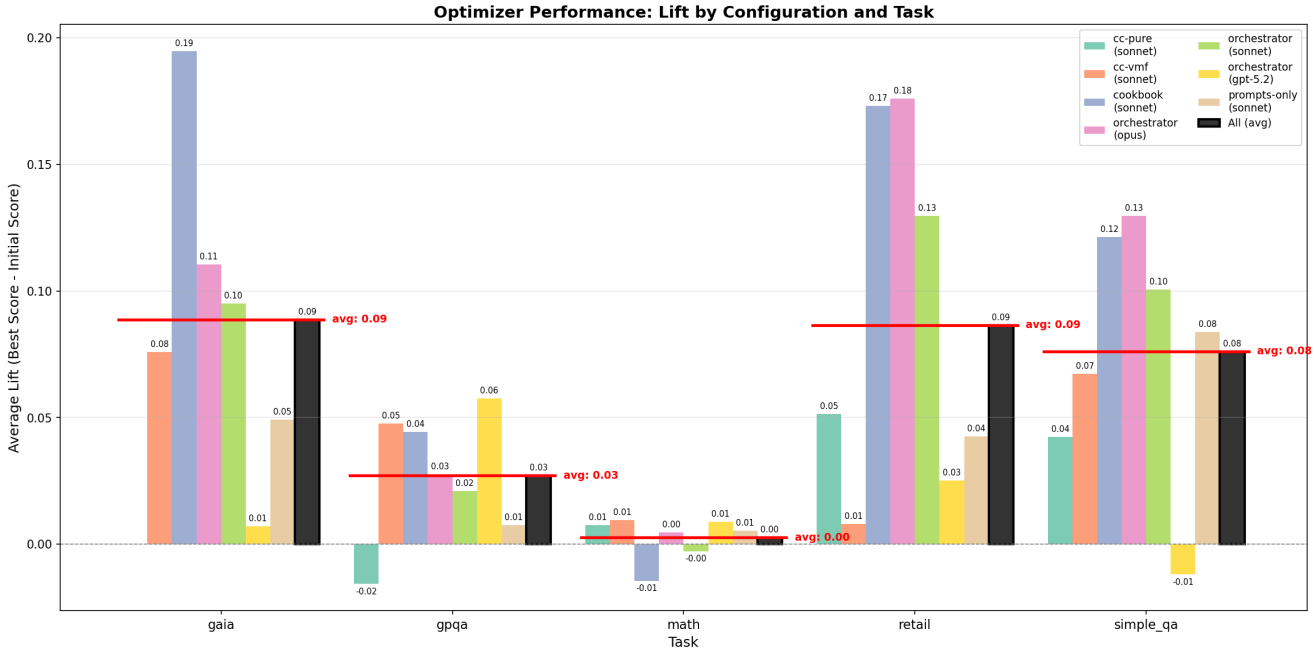


Figure 5: Optimizer performance across tasks. Each grouped bar shows the average lift (best score minus initial score) for each optimizer configuration. Red horizontal lines indicate the mean lift per task across all configurations.

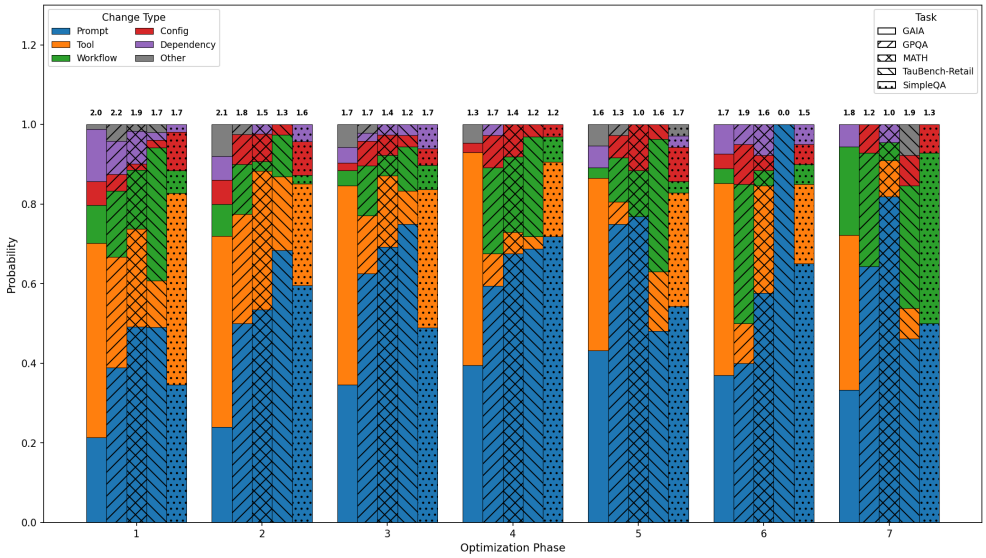


Figure 6: We plot the probability of changes made between successive evaluations in a trajectory falling into one of several types (e.g. prompt, tool, workflow changes) across all trajectories per task. **Bold** numbers capture the entropy of the probability distribution defined by each bar.

A.2.3 Semantic Interpretability

Git versioning of modifications to the target agent **A**, tracing of the execution of the optimizer agent S, and tracking of target agent evaluations enables us to uncover semantic patterns in the optimization trajectories and correlate them with target agent performance.

Phase Subtypes. In Figure 8, we show results from the phase tagging procedure described in the main body of the paper. Here we additionally extract sub-types for each of the main change types mentioned earlier. We present the

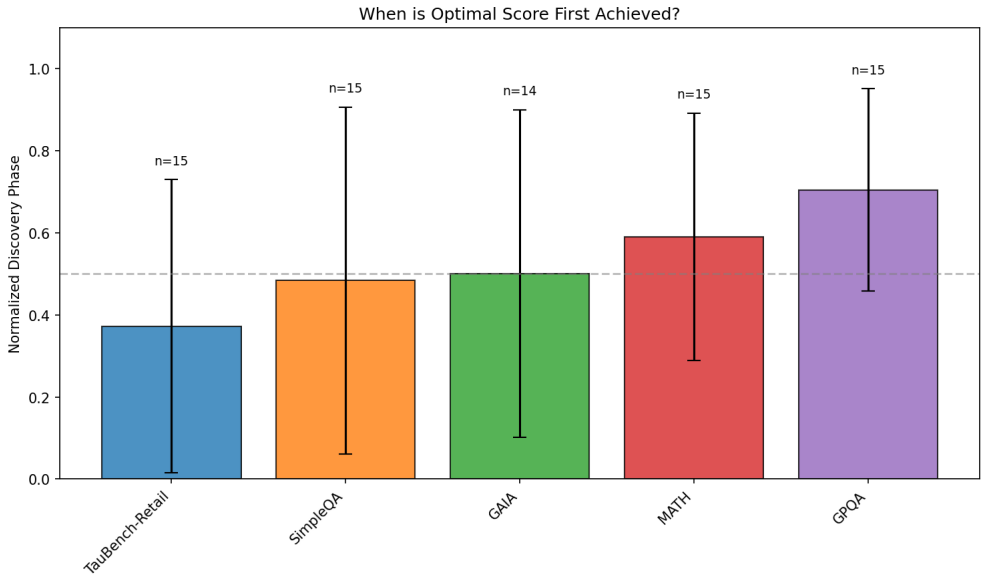


Figure 7: Mean normalized phase at which the optimal commit is discovered across trajectories. We average over all VERO trajectories conducted for the benchmark study.

sub-types for prompt, tool, and workflow changes for the 3 VERO variants. We note that prompts and workflows are dominated by the top subtypes in those respective bars, whereas for tools, the proportion of "other" subtypes is more substantial. This suggests that tool design may be a source of diversity in the changes LLMs make to other agents.

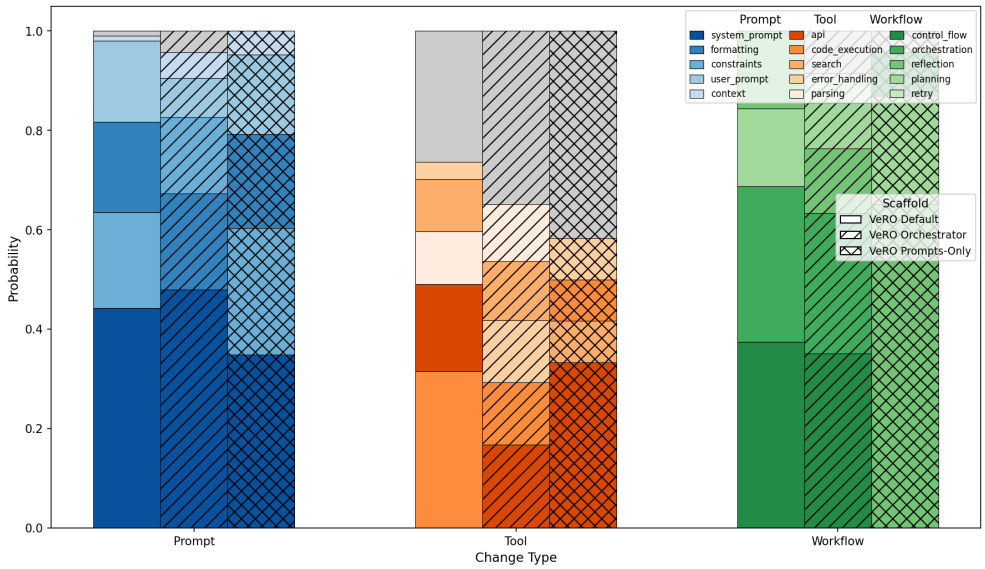


Figure 8: Top sub-types for 3 types of modifications optimizers make to targets.

Diff Embeddings Show Clusters. Figure 10 captures the semantic content of the final commits produced by optimization runs on each of the 5 benchmark tasks. We extract the final commit in each optimization phase in each trajectory and compute `diffs` with respect to an empty commit. We then embed these `diffs` using an embedding model, OpenAI's `text-embedding-3-large`, and project these embeddings into 2-dimensional space using UMAP. Colors of points depict which phase of the trajectory they occur in, with darker points depicting earlier phases. Initial points form discernible clusters as one might expect; variation exists because our initial commits, though the

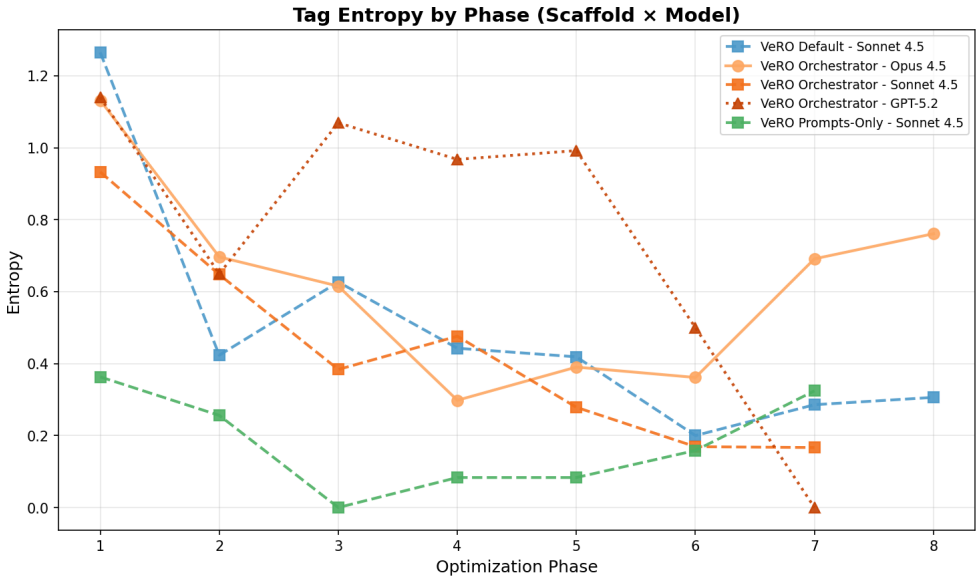


Figure 9: Entropy of change types as a function of optimization phase for different scaffold-variant-model triples.

same for the subset of code relevant to the task, contain other code that alter their embeddings. Interestingly, we also find varying levels of clustering of final commits; SimpleQA and TAU-Bench Retail show two very distinct final commit clusters, suggesting that despite different configurations, optimizer solutions are not that semantically diverse. Additionally, we observe that many orange and red points, i.e. points in intermediate phases are not visible in the graphs; this is because they are hidden behind the final commits, suggesting another interesting feature of these trajectories: the main shape of the solution is often found fairly quickly, with subsequent changes being relatively small. This could be connected to the drop in entropy we observe in change type tags as a function of phase.

A.2.4 Example Trajectories

We include several explicit examples of optimization trajectories. For each trajectory, we show train, validation, and test scores where available. We also show short summaries extracted via the same procedure as our change type tag extraction process described in the main body of the paper.

We bring attention especially to Figure 11 where we observe an interesting combination of both tool and prompt changes that seem intuitively helpful for the MATH task. In fact, we do see a large improvement in validation score ($0.78 \rightarrow 0.92$), though this is not reflected in the test score for the same dataset and is also associated with a lower score on the training set. Note that the test set is only evaluated with the initial commit and the commit with the highest validation score, hence, only two points are shown.

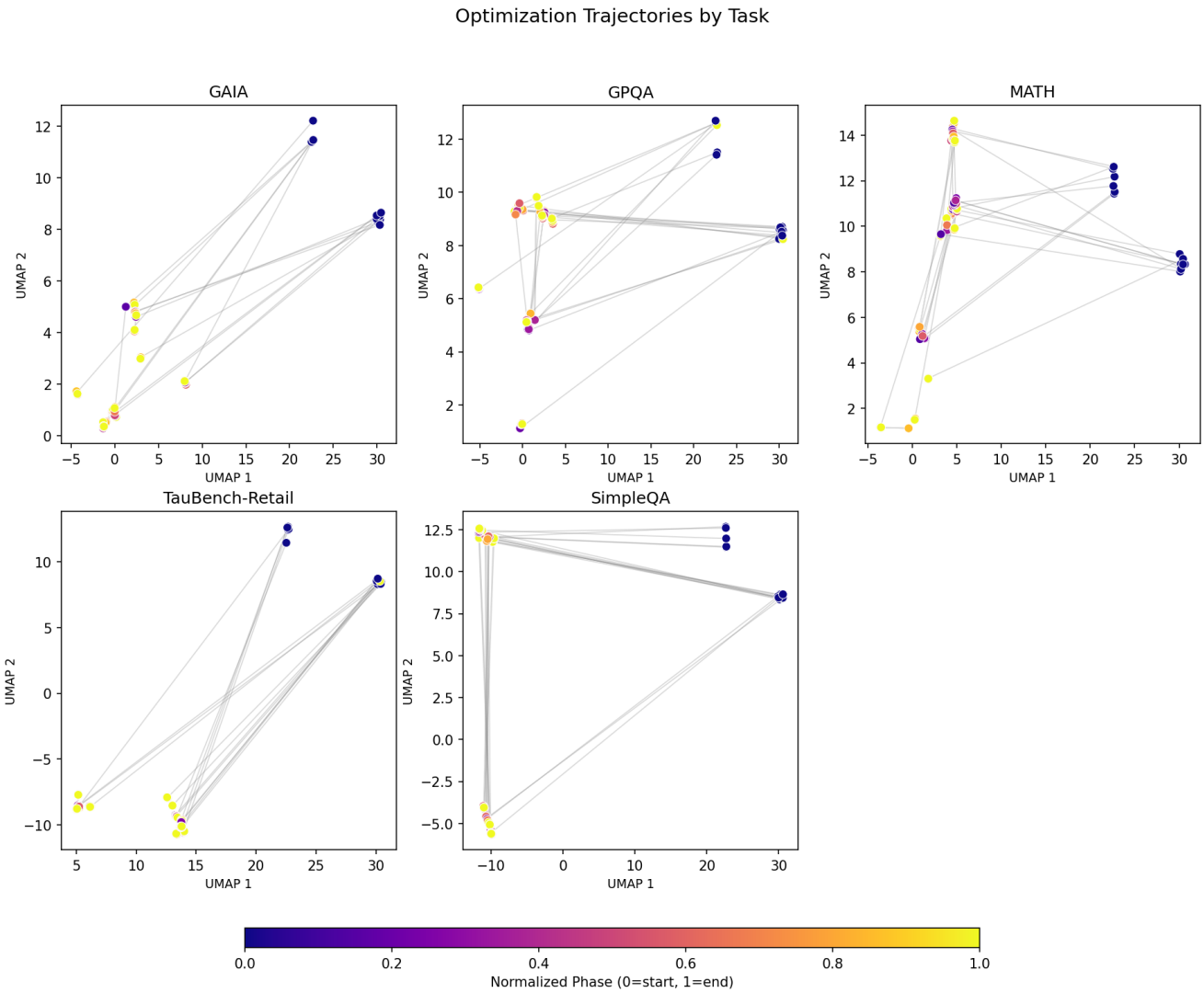


Figure 10: UMAP-projected text embeddings of Git `diffs` of commits made in each optimization trajectory with respect to an empty commit. Initial and final commits show natural clusters.

Toolset	Capabilities	Core Abstractions
File System Tools		
`FileRead`	Read files with line ranges and pagination	`Filesystem`
`FileWrite`	Write/edit files with search-replace; auto-commits changes	`Filesystem`, `GitWorktree`
`BashTool`	Restricted shell commands for filesystem search (`ls`, `find`, `tree`, `pwd`)	`Filesystem`
`Grep`	Regex search using ripgrep with multiple output modes	`Filesystem`
Git Tools		
`GitViewer`	Inspect diffs, commit log, current/base commits	`GitWorktree`
`GitControl`	Restore to previous commits (preserving history)	`GitWorktree`
Experiment Tools		
`ExperimentRunner`	Run evaluations on dataset splits up to a specified number of times	`Evaluator`, `ExperimentDatabase`
`ExperimentViewer`	Browse experiment results and execution traces	`ExperimentDatabase`
`DatasetViewer`	View dataset samples and metadata	`Dataset`
Planning & Context		
`TodoList`	Task management with status tracking	—
`ContextStore`	Key-value store for text artifacts with versioning	—
`think`	Explicit reasoning step via a placeholder tool	—
Web Tools		
`WebSearch`	Web search via Serper API	—
`WebFetch`	Fetch and extract text from web pages/PDFs	—
Sub-agents		
`Call Sub-Agent`	Invoke a sub-agent with a dynamic set of tools	—
Resource Management		
`ResourceControl`	Discover and edit `@resource`-decorated functions/classes via AST	`Filesystem`

Table 4: Toolsets available in the VERO scaffold. Each toolset is tied to core VERO abstractions.

Table 5: Optimizer configurations evaluated. Toolset changes are relative to the base scaffold. The Cookbook is a pre-loaded *ContextStore* containing agent design patterns.

Scaffold	Variant	Toolset Configuration
VERO	Default	Base toolset: *FileRead*, *FileWrite*, *Grep*, *BashTool*, *GitViewer*, *GitControl*, *ExperimentRunner*, *ExperimentViewer*, *DatasetViewer*, *WebSearch*, *WebFetch*, *TodoList*, *think*, *ContextStore* (Cookbook). Sub-agent delegation enabled.
VERO	Orchestrator	Orchestrator tools restricted to: *ExperimentRunner*, *GitControl*, *GitViewer*, *ContextStore*, *TodoList*, *think*. Removes file read/write tools from orchestrator to force sub-agent usage. Sub-agents retain full toolset; orchestrator delegates code tasks.
VERO	Resources-Only	Removes *FileWrite*; adds *ResourceControl*. Optimizer can only modify `@resource`-decorated functions (prompts, tool descriptions, parameters). No direct file writes or shell mutations.
Claude Code	VERO Tools	Base: Claude Code native tools (file editing, Bash, web search). Adds *ExperimentRunner*, *ExperimentViewer*, *DatasetViewer*, *ContextStore* via MCP. Auto-commit hook instruments file writes for observability.
Claude Code	Pure	Unmodified Claude Code. No VERO tools exposed. Optimizer must invoke the target agent via shell and parse outputs manually. No auto-commit instrumentation.

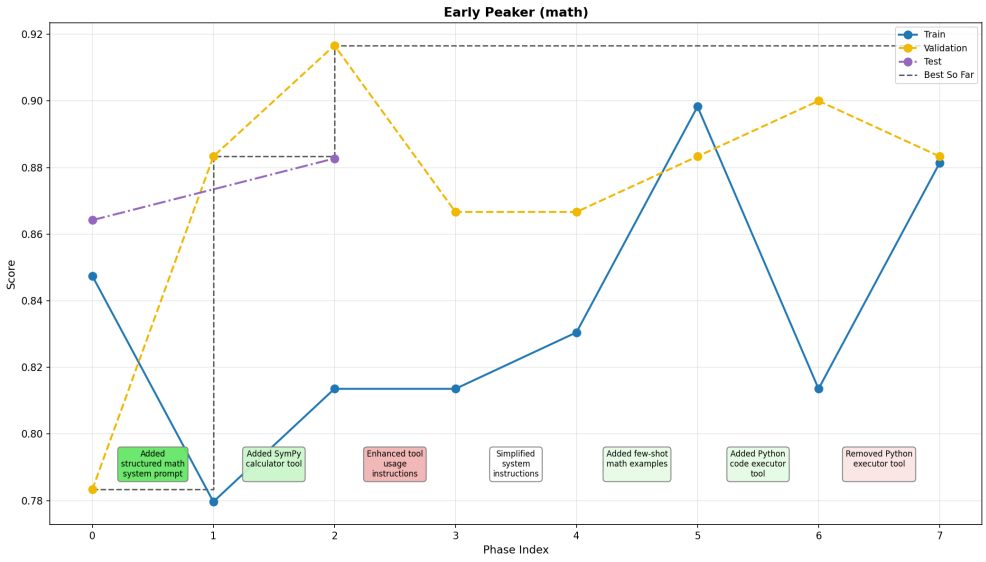


Figure 11: An example trajectory of changes made by VERO *Orchestrator* with Sonnet 4.5 on the MATH task.

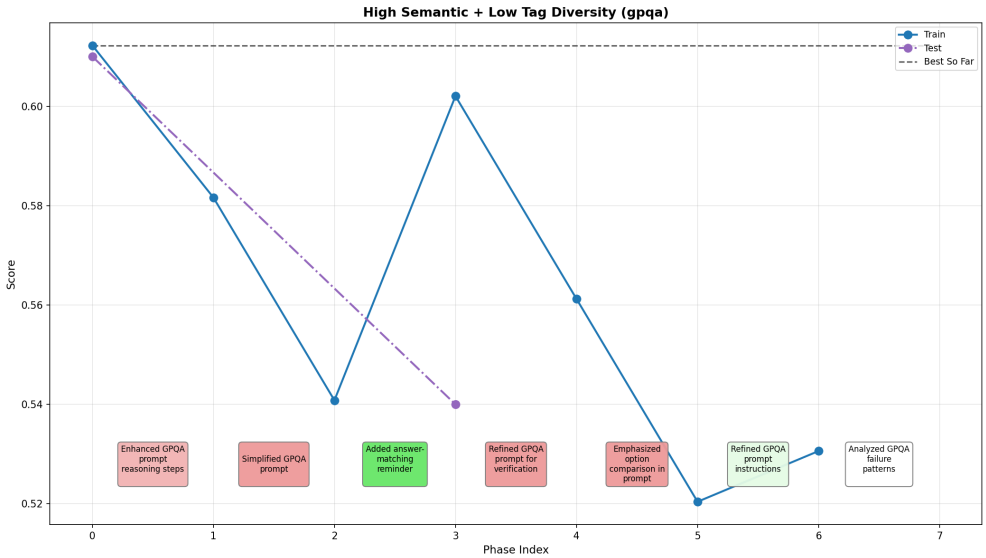


Figure 12: An example trajectory of changes made by VERO *Resources Only* with Sonnet 4.5 on the GPQA task.

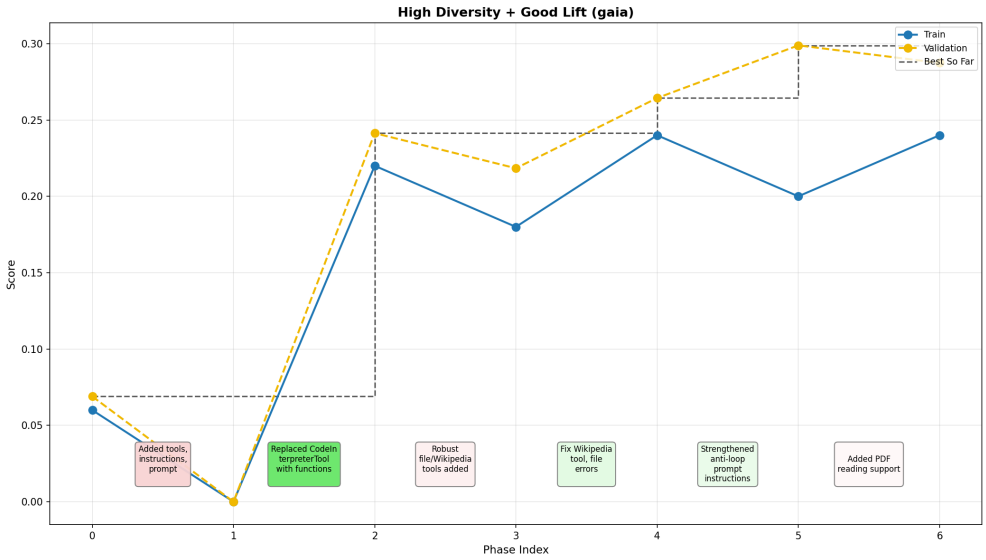


Figure 13: An example trajectory of changes made by VERO with Sonnet 4.5 on the GAIA task.

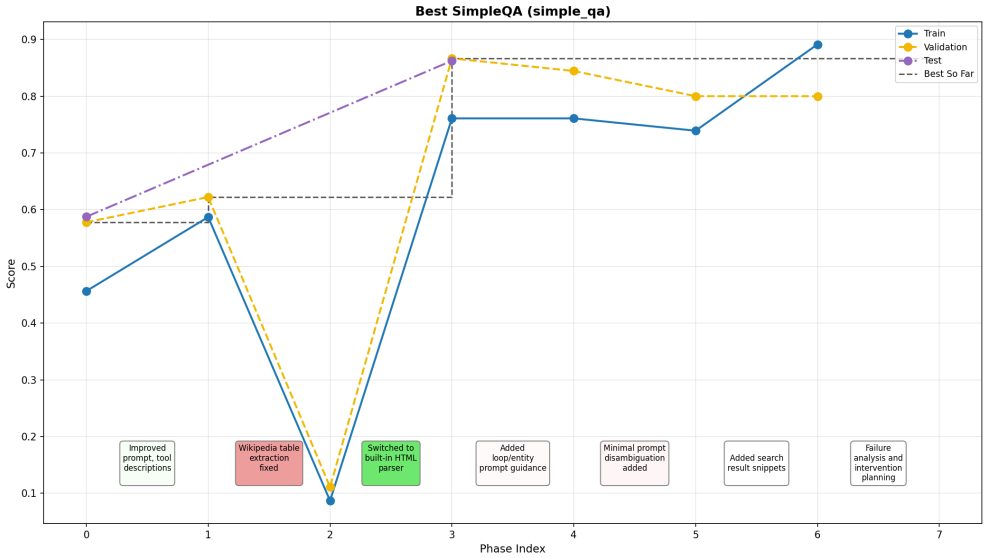


Figure 14: An example trajectory of changes made by VERO *Orchestrator* with Opus 4.5 on the SimpleQA task.

A.3 Case Study: Supplementary Materials

Here, we provide several supplementary details and analyses pertaining to our case study on optimization of realistic agents (section: 5.3).

Agent	GAIA	FACTS	SimpleQA
Pawn	19.5%	54.7%	68.9%
Knight	43.7%	64.7%	84.4%

Table 6: Baseline accuracy for case study agents.

Agent	Template	Mean	Std Dev
Pawn	Cookbook+Reasoning	57.9%	6.79%
	Minimal	53.7%	5.15%
	Tool-Centric	55.8%	0.93%
	Evidence-Based	53.3%	1.05%
Knight	Cookbook+Reasoning	66.6%	2.08%
	Minimal	68.7%	1.45%
	Tool-Centric	68.0%	1.64%
	Evidence-Based	66.6%	0.88%

Table 7: Mean accuracy and standard deviation on FACTS across templates (4 iterations each).

Template	Knight (s)	Pawn (s)
Cookbook+Reasoning	30.6	14.5
Minimal	56.0	20.4
Tool-Centric	56.6	32.8
Evidence-Based	**26.2**	**12.6**

Table 8: Mean runtime (seconds) per sample by template.

A.3.1 Agent Comparison: Pawn vs. Knight

Table 9 details the architectural differences between the two target agents used in the case study. These agents were designed to represent opposite ends of the capability spectrum while sharing the same underlying LLM (GPT-4.1 mini).

Design rationale. Pawn represents the "blank slate" scenario: a functional but minimal agent that optimizers can expand through tool additions, prompt engineering, and architectural changes. Knight represents the "refinement" scenario: an agent already incorporating best practices from the GAIA leaderboard, testing whether optimizers can identify remaining inefficiencies in sophisticated workflows.

A.3.2 Instruction Template Comparison

Table 10 provides a detailed feature comparison of the four instruction templates used to guide the optimizer.

Template design philosophy.

Aspect	Pawn (Minimal)	Knight (Sophisticated)
Tools		
Total tools	4	6
Web search	Basic (3 results, no retries)	Enhanced (5+ results, retry logic)
Wikipedia	✗	✓
Reflect tool	✗	✓ (LLM-powered self-evaluation)
File reading	Text only (.txt, .md, .json, .csv)	Complex formats (PDF, Excel, Word, ZIP)
Python execution	✓	✓
Page fetch	✓	✓
Python Environment		
Available libraries	math, datetime, json, re	math, datetime, pandas, numpy, statistics, json, re
Package count	3	12+
Agent Configuration		
System prompt	25 lines (minimal instructions)	140 lines (CoT, ReACT, detailed guidance)
Max turns	20	40
Reasoning patterns	None	Chain-of-Thought, ReACT, self-verification

Table 9: Architectural comparison of Pawn and Knight agents.

Feature	Cookbook+Reasoning	Minimal	Tool-Centric	Evidence-Based
Lines of code	398	140	396	503
Cookbook access	✓	✗	✓	✓
Tool docstring guidance	✗	✗	✓ (detailed)	Tiered hierarchy
LLM-integrated tool examples	✗	✗	✓	✗ (discouraged)
Planning examples	✓	✗	✓	✓
Reasoning patterns	12 patterns	✗	✓	Condensed
Anti-patterns section	1 section	✗	✗	6 explicit patterns
Failure categorization	✗	✗	✗	5 categories
Reversion protocol	✗	✗	✗	✓
Sub-agent delegation	✓	✓	✓	✓

Table 10: Feature comparison of optimizer instruction templates.

- **Cookbook+Reasoning** provides comprehensive guidance including a library of optimization patterns (e.g., "add verification steps," "implement retry logic") and a structured 4-phase workflow (Analyze → Plan → Implement → Evaluate).
- **Minimal** ablates prescriptive guidance, testing whether optimizers perform better with creative freedom. At 35% the size of other templates, it provides only core orchestration instructions.
- **Tool-Centric** emphasizes tool-level improvements: detailed docstring examples, patterns for LLM-integrated tools (reflect, summarize, classify), and guidance on tool parameter design.
- **Evidence-Based** incorporates learnings from the other templates: a 3-tier optimization hierarchy (Tools > Workflow > Prompts), 6 explicit anti-patterns with examples, a 5-category failure framework, and single-variable experimentation constraints.

A.3.3 Optimization Trajectory Analysis

Tables 11 and 12 provide complete per-iteration results. Each worktree name corresponds to a Git branch containing the optimizer's modifications.

Template	Iter	Worktree	FACTS	GAIA	SimpleQA	Time (s)
Baseline	–	–	64.72%	43.68%	84.44%	26.6
Cookbook+Reasoning	1	plain_pond	65.06%	43.68%	84.44%	27.2
	2	noisy_block	65.73%	**49.43%**	84.44%	26.3
	3	solitary_sky	65.84%	45.98%	86.67%	26.3
	4	quiet_sunset	**69.66%**	48.28%	84.44%	45.7
Minimal	1	yellow_bar	67.98%	**50.57%**	84.44%	39.7
	2	shiny_night	**70.34%**	47.13%	**88.89%**	107.8
	3	purple_brook	67.64%	47.13%	82.22%	36.5
	4	orange_wind	N/A*	43.68%	84.44%	31.5
Tool-Centric	1	patient_unit	66.74%	49.43%	84.44%	45.6
	2	dawn_hat	67.98%	47.13%	84.44%	49.5
	3	blue_night	66.97%	44.83%	84.44%	47.0
	4	spring_star	**70.34%**	43.68%	86.67%	73.1
Evidence-Based	1	ancient_haze	66.85%	45.98%	84.44%	26.2
	2	misty_pine	67.19%	45.98%	**88.89%**	24.2
	3	empty_union	65.28%	45.98%	86.67%	26.8
	4	old_bird	67.08%	43.68%	86.67%	27.0

Table 11: Knight agent: per-iteration results. Bold indicates best result per template. *Evaluation crashed

Template	Iter	Worktree	FACTS	GAIA	SimpleQA	Time (s)
Baseline	–	–	54.72%	19.54%	68.89%	9.0
Cookbook+Reasoning	1	proud_disk	62.36%	29.89%	68.89%	14.2
	2	quiet_glitter	54.83%	25.29%	71.11%	13.7
	3	black_lake	49.33%	25.29%	51.11%	10.7
	4	polished_band	**65.17%**	**31.03%**	**82.22%**	21.9
Minimal	1	snowy_leaf	48.76%	17.24%	51.11%	17.6
	2	little_silence	54.83%	**28.74%**	**73.33%**	31.4
	3	raspy_mouse	**60.56%**	22.99%	71.11%	20.1
	4	ancient_waterfall	50.79%	22.99%	66.67%	17.0
Tool-Centric	1	gentle_pine	55.73%	27.59%	68.89%	30.9
	2	winter_dream	**56.85%**	**29.89%**	71.11%	32.6
	3	throbbing_snowflake	54.61%	24.14%	**75.56%**	28.3
	4	fancy_truth	55.96%	19.54%	62.22%	32.4
Evidence-Based	1	calm_truth	54.04%	19.54%	60.00%	12.7
	2	noisy_fire	52.70%	24.14%	66.67%	11.2
	3	white_tooth	**54.27%**	**27.59%**	66.67%	12.9
	4	sparkling_hat	52.13%	**27.59%**	62.22%	10.8

Table 12: Pawn agent: per-iteration results. **Bold** indicates best per template; red indicates regression below baseline.

A.3.4 Notable Optimization Patterns

Analysis of the commit histories reveals several recurring patterns:

High-variance iterations.

- **black_lake** (Pawn, Cookbook+Reasoning iter3): Added a complex multi-step verification tool that improved GAIA (+5.75pp) but catastrophically degraded SimpleQA (−17.78pp). The tool introduced unnecessary overhead for simple factual queries.
- **shiny_night** (Knight, Minimal iter2): Achieved peak FACTS accuracy (70.34%) and SimpleQA (88.89%) but with 4× baseline runtime (107.8s). The optimizer added aggressive retry logic and multiple verification passes.
- **snowy_leaf** (Pawn, Minimal iter1): First iteration regressed on all three metrics, demonstrating that minimal guidance can lead to harmful modifications when the optimizer lacks domain knowledge.

Stable progressions.

- **Evidence-Based template** (both agents): All iterations maintained similar runtime to baseline (±3s), reflecting the template's emphasis on lightweight modifications. Variance was lowest across all templates.
- **polished_band** (Pawn, Cookbook+Reasoning iter4): Best single iteration for Pawn, achieving simultaneous improvements on all three metrics (+10.45pp FACTS, +11.49pp GAIA, +13.33pp SimpleQA). Inspection reveals the optimizer added Wikipedia search and improved error handling.

Template-specific behaviors.

- **Cookbook+Reasoning**: Produced the most diverse modifications, including new tools, architectural changes, and prompt rewrites. High variance but highest peaks for Pawn.
- **Minimal**: Encouraged creative exploration, leading to both breakthrough results (shiny_night) and failures (snowy_leaf). Best for Knight, worst for Pawn.
- **Tool-Centric**: Focused modifications on tool docstrings and parameters. Moderate performance with moderate variance.
- **Evidence-Based**: Constrained to incremental changes. Best runtime efficiency and lowest variance, but prevented breakthrough modifications.

Iteration dynamics.

- Early iterations (1–2) often showed the largest GAIA improvements, suggesting low-hanging fruit in the optimization landscape.
- Later iterations (3–4) tended to improve FACTS but sometimes regressed GAIA, indicating potential overfitting to the broader FACTS distribution.
- No template consistently improved across all iterations; diminishing returns appeared after iteration 2–3.

A.3.5 Baseline Performance Details

Agent	Dataset	Accuracy	Correct/Total	Avg Time (s)	Errors
Knight	FACTS	64.72%	576/890	29.2	60
	GAIA	43.68%	38/87	42.1	6
	SimpleQA	84.44%	38/45	8.5	0
Pawn	FACTS	54.72%	487/890	8.1	17
	GAIA	19.54%	17/87	13.6	3
	SimpleQA	68.89%	31/45	5.1	0

Table 13: Baseline performance for case study agents before optimization.

A.4 Limitations

We identify four categories of limitations in our current evaluation methodology.

Budget definition is incomplete. We define the optimization budget, B, as the number of *ExperimentRunner* invocations, where each invocation evaluates the target agent on the full training or validation set. However, this metric does not capture the computational cost of each optimization phase; the tokens consumed by the optimizer, the number of tool calls, or the API credits expended between evaluations. Two optimizers with identical $n_{\mathcal{E}}$ may differ substantially in total compute. Future work should incorporate multi-dimensional budget constraints (e.g., token limits, wall-clock time, API cost) to enable fairer comparison and better reflect deployment constraints.

Budget sensitivity is unexplored. Our choice of $B = 8$ for the benchmark study and $B = 5$ for the case study was pragmatic rather than principled. It remains unclear whether larger budgets would yield continued improvement or exhibit diminishing returns. Furthermore, we do not distinguish between a single optimizer session with B evaluations versus B independent single-evaluation sessions. These may produce qualitatively different optimization trajectories due to differences in context accumulation and exploration strategy. Systematic ablation of budget size and structure is needed to characterize the sample efficiency of different optimizer configurations.

Human baselines are absent. We lack human performance data on the agent optimization task for most target agents in our benchmark. While public leaderboards exist for end-task performance (e.g., GAIA), these reflect human-crafted agents developed without the constraints we impose (fixed budget, restricted search space). Establishing human baselines where expert developers optimize target agents under VERO's protocol would provide an upper bound for interpreting optimizer performance and quantifying the gap between current systems and human capability.

External API variability introduces noise. Several target agents rely on public APIs (Wikipedia, web search) whose responses change over time and may be temporarily unavailable. This temporal variability injects noise into evaluations: an agent that succeeds today may fail tomorrow due to altered search results rather than any change in agent quality. Additionally, we do not currently guard against **reward hacking** through memorization. Optimizers could potentially hard-code answers found in public replicas of benchmark datasets. Sandboxing external API calls or using cached snapshots would improve reproducibility; detecting and penalizing memorization requires further methodological development.

A.5 Future Work

The findings and limitations of this work suggest several directions for future research.

Structured exploration of the program space. Current optimizers operate via sequential modification: each candidate agent is conditioned on the full history of prior candidates through the LLM's context window. This amounts to depth-first search in the space of programs. Tree search methods branching from promising candidates, allocating budget across parallel trajectories may explore more efficiently. However, the program space is infinite, and the space of `diffs` is similarly unbounded, making direct application of methods like Monte Carlo Tree Search non-trivial. One approach is to learn finite-dimensional representations of code (via embeddings) and modifications (via high-level feature classifiers, e.g., "adds tool", "modifies prompt", "introduces verification step"), then search over this compressed space. Budget allocation across different modification types (prompt vs. tool vs. architecture) is an open question with practical implications for optimizer design.

Principled construction of optimizer context. Our case study reveals that instruction templates significantly affect optimization outcomes yet we lack theory for designing effective templates. What information should cookbooks contain? How should patterns be organized and retrieved? Can we automatically construct or refine cookbooks based on optimization trajectories? Meta-learning approaches that improve optimizer instructions from experience could yield compounding gains.

Self-improving optimizers. If agents are code and coding agents produce code, then coding agents could, in principle, improve themselves. Applying VERO to optimize its own optimizer scaffolds, or the coding agents that use them, opens the possibility of recursive self-improvement. This connects to broader questions about open-ended evolution and the stability of self-modifying systems. Initial experiments could target narrow capabilities (e.g., improving the optimizer's ability to diagnose failures) before attempting full self-optimization.

Integration with reinforcement learning. VERO's structured traces and versioned snapshots provide the reward signal and state representation needed for RL. Training LLMs on optimization trajectories, rewarding successful modifications and penalizing regressions, could improve base model capabilities on the agent optimization task. The benchmark suite provides a ready environment; the primary challenges are credit assignment (which modification caused improvement?) and sample efficiency (optimization trajectories are expensive to generate).